

模块划分说明

制定日：2023-04-01 / 最终修订：2025-03-20 / 管理：QA 基盘团队

ATP 的模块字典固定 8 个，**不能自行增删**。本篇说明每个模块的边界，以及边界模糊时的归属判断规则。

模块归属直接决定 case_code 的前缀，改起来要连编号一起改，所以新建案例时值得先想清楚。

为什么固定 8 个

模块字典设计成只读表，不开放自定义，理由是：

1. ****模块是统计口径**** —— 覆盖率、通过率、缺陷密度都按模块聚合。口径可变的统计没有可比性
2. ****防止无限细分**** —— 允许自定义时，实际发生的是每个小组建自己的模块，最后有几十个模块、每个模块几条案例，统计失去意义
3. ****强制归类的思考**** —— 一个功能属于哪个模块想不清楚，往往意味着这个功能本身的职责就不清晰

代价是有些案例确实卡在边界上。下面的判断规则就是为这些情况准备的。

八个模块的边界

M001 LOGIN — ログイン / 登录认证

****包含****：登录、登出、密码重置、账号锁定、双因素认证、会话超时、记住登录状态、SSO 跳转

****不包含****：

- 用户信息修改 `USER`
- 用户的创建 / 停用 `ADMIN`
- 权限不足时的拦截 按****被拦截的那个功能****所在模块归类

最后一条是最常被问的。「普通用户访问管理后台被拒绝」这条案例属于 `ADMIN` 而不是 `LOGIN` —— 因为它测的是后台的权限控制，不是登录本身。

M002 SEARCH — 検索 / 商品搜索

****包含****：关键词搜索、筛选、排序、分页、搜索结果为空、搜索建议 / 自动补全、搜索历史

****不包含****：

- 商品详情页 详情页本身没有独立模块，归 `SEARCH`（它是搜索结果的下游）
- 后台的商品管理 `ADMIN`
- 订单列表的筛选 `ORDER`

注意区分「面向顾客搜索」和「后台的列表查询」。后者归各自的业务模块。

M003 CART — カート / 购物车

****包含****：加入购物车、修改数量、删除商品、购物车合计计算、库存不足提示、优惠券应用、跨设备同步

****不包含****：

- 结算页之后的流程 `ORDER`
- 支付 `PAYMENT`

分界点是**点击「去结算」的那一刻**。之前属于 `CART`，之后属于 `ORDER`。

M004 ORDER — 注文 / 订单管理

****包含****：下单流程、订单列表、订单详情、取消订单、退货申请、配送信息填写、订单状态流转

****不包含****：

- 支付环节 `PAYMENT`
- 后台的订单处理 `ADMIN`

一条完整的「下单并支付」流程横跨 `ORDER` 和 `PAYMENT`。按下面的「主要验证点」规则判断。

M005 USER — ユーザー管理 / 用户中心

****包含****：个人信息查看 / 修改、收货地址管理、修改密码（已登录状态）、消息通知设置、积分 / 会员等级

****不包含****：

- 忘记密码的重置流程（未登录状态）`LOGIN`
- 管理员操作他人账号 `ADMIN`

区分点是**操作者与被操作对象是否为同一人**。用户改自己的信息是 `USER`，管理员改别人的是 `ADMIN`。

M006 PAYMENT — 決済 / 支付

****包含****：支付方式选择、支付流程、支付失败处理、退款、发票开具、分期付款

****不包含****：

- 下单流程本身 `ORDER`
- 优惠券的应用 `CART`（优惠在购物车阶段计算）

M007 REPORT — レポート / 报表导出

****包含****：报表生成、条件筛选、导出（CSV / Excel / PDF）、大数据量的异步导出、导出权限控制、定时报表

****不包含****：

- 页面上的数据展示 归各自业务模块
- 报表相关的后台配置 `ADMIN`

分界点是****是否产生可下载的文件****。屏幕上看到的是业务模块，能导出成文件的是 `REPORT`。

M008 ADMIN — 管理画面 / 后台管理

****包含****：后台的所有功能 —— 用户管理、商品管理、订单处理、系统配置、权限管理、操作日志、数据字典维护

****判断规则****：****只要是在管理后台界面里操作的，一律归 `ADMIN`****，不管操作对象属于什么业务域。

这是唯一一个按「界面」而非「业务域」划分的模块。这么做是因为后台的操作者、权限模型、UI 风格都和前台完全不同，混在一起会让统计失去意义。

边界判断规则

案例跨多个模块时，按以下顺序判断：

规则一：看主要验证点

案例的断言集中在哪个模块的功能上，就归哪个模块。

- > 「下单后跳转到支付页，完成支付，验证订单状态变为已支付」
- > 断言的是****订单状态****，归 `ORDER`
- > 「下单后在支付页选择信用卡，验证支付失败时的错误提示」
- > 断言的是****支付的错误处理****，归 `PAYMENT`

同样的前置流程，验证点不同，归属就不同。

规则二：看最后一个操作步骤

主要验证点也判断不清时，看案例最后停在哪个模块的界面上。

规则三：看操作界面（后台优先）

在管理后台里操作的，一律 `ADMIN`，此规则优先级高于前两条。

- > 「管理员在后台把订单状态改为已发货，验证状态更新成功」

> 虽然验证点是订单状态，但操作在后台，归 `ADMIN`

规则四：拆分

三条规则都判断不清，说明这条案例**测了太多东西**。按模块拆成多条。

这不是回避问题 —— 一条横跨三个模块的案例，失败时本来就难以定位，拆开对排查也有好处。

归属错了怎么办

case_code 的模块段一旦生成就无法修改。归属错误时的处理：

1. **发现得早（还没被引用）** 废弃原案例，在正确模块下重建
2. **已经被缺陷单或报告引用** 保留原案例，在 `description` 里注明正确归属

不要为了修正归属而修改历史 case_code，那会让既有引用全部失效。归错模块的代价（统计口径略有偏差）远小于引用断裂的代价。